

C implementation Double-Ended Queue

Last Updated : 23 May, 2024

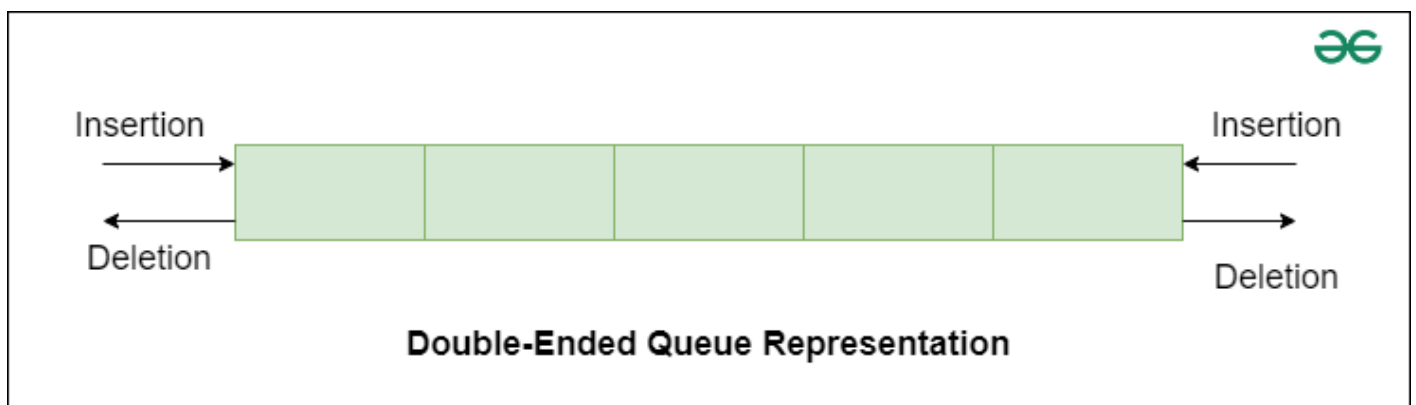


The double-ended queues, called deques for short, are a generalized form of the queue. It is exactly like a queue except that it does not follow the FIFO rule (First in first out) so, the elements can be added to or removed from either the front(head) or back(tail) of the deque.

In this article, we will learn about the double-ended queue implementation in C. We will also look at the working of the double-ended queue and the basic operations that can be performed using the double-ended queue in C.

Double-Ended Queue Representation in C

In a [deque](#) we maintain two pointers front and rear, that point to either end of the deque. The elements in a deque extend from the front end to the rear end and since it is circular, `dequeue[n-1]` is followed by `dequeue[0]`.



Representation of Deque

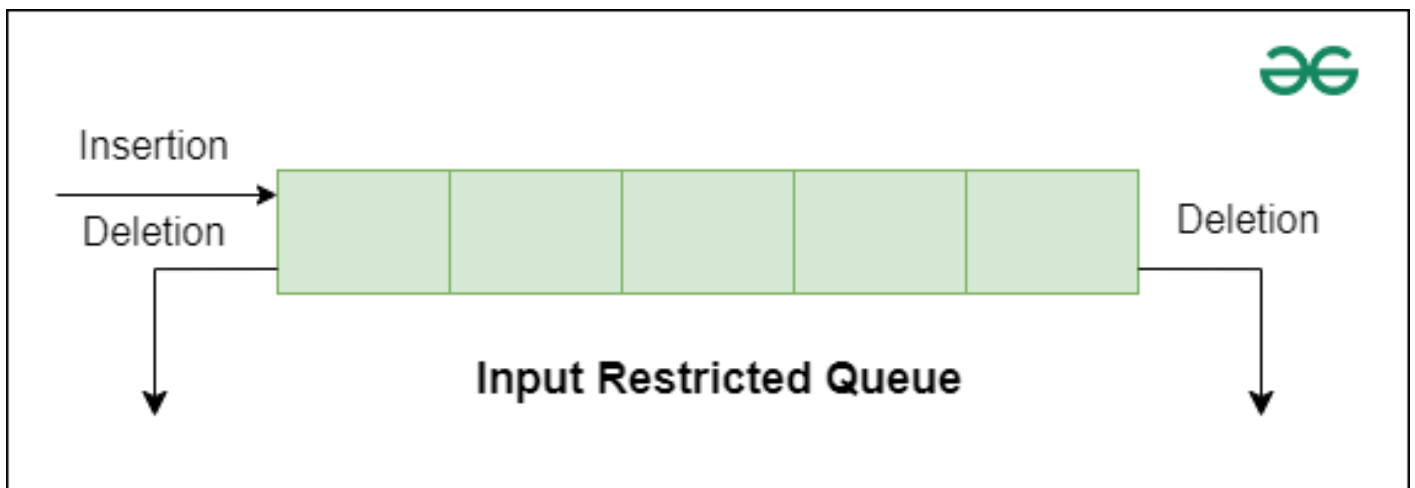
Types of Deque

There are two variants of a double-ended queue. They include :

- Input restricted queue
- Output restricted queue

Input Restricted Queue

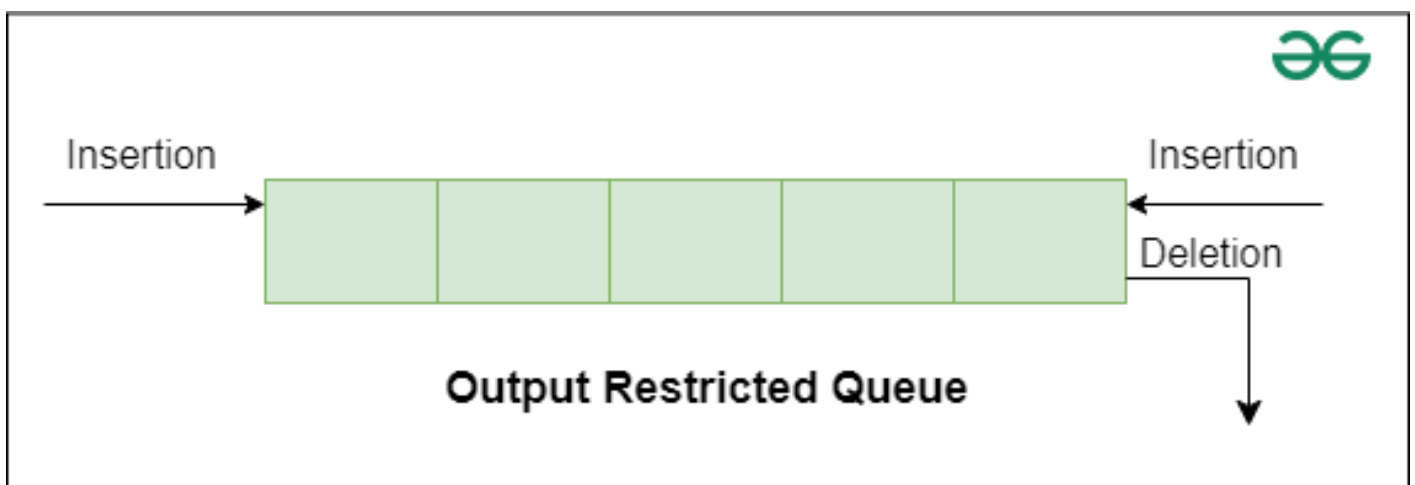
In Input restricted double ended queue, insertions can be done only at one of the end, while deletions can be done from both ends.



Input Restricted Deque

Output Restricted Queue

In output restricted double ended queue, deletions can be done only at one of the end, while insertions can be done on both ends.



Output Restricted Queue

Implementation of Double-Ended Queue in C

Dequeues can be implemented using two data structures.

- [Implementation of Deque using circular array](#)
- [Implementation of Deque using doubly linked list](#)

Here, we will see implementation of deque using circular array. Circular array is an array whose last element is connected to first element this creates a circular structure. In circular array if an array is full we again start from the beginning unlike linear array which throws an overflow message.

Basic Operations on C Double-Ended Queue

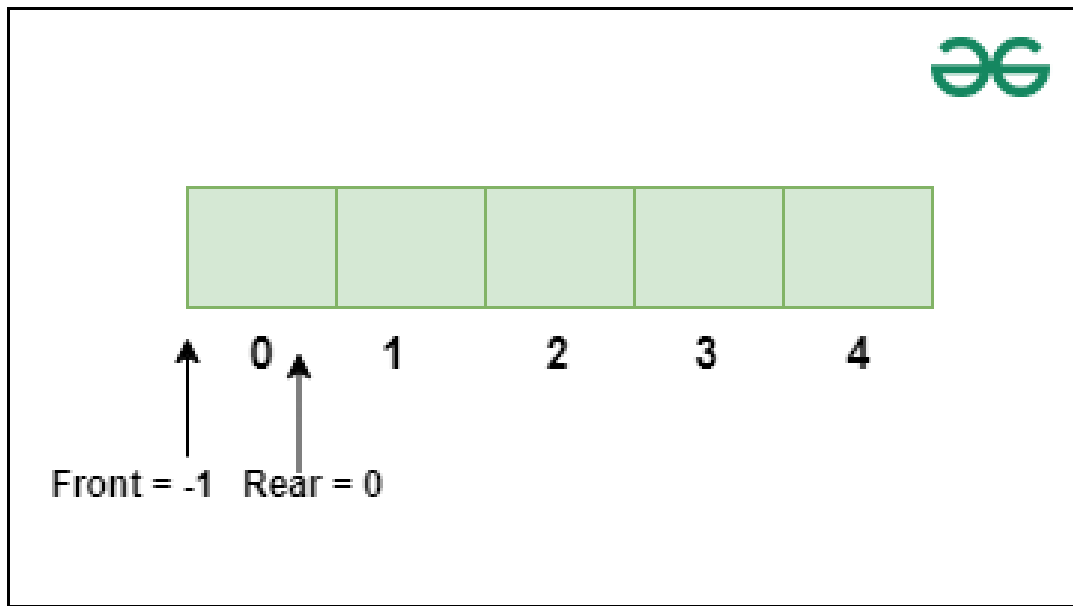
We can perform the following basic operations in a double-ended queue:

Operation	Description	Time Complexity	Space Complexity
Insertion at front	This function is used to insert an element at the front end of the deque.	$O(1)$	$O(1)$
Insertion at rear	This function is used to insert an element at the rear end of the deque.	$O(1)$	$O(1)$
Deletion at front	This function is used to delete an element from the front end of the deque.	$O(1)$	$O(1)$
Deletion at rear	This function is used to delete an element from the rear end of the deque.	$O(1)$	$O(1)$
Check empty	This function is used to check if the deque is empty or not.	$O(1)$	$O(1)$
Check full	This function is used to check if the deque is full or not.	$O(1)$	$O(1)$

Here, **front** denotes the first element of the deque and **rear** denotes the last element of the deque.

Before performing these operations, we must follow the below two steps:

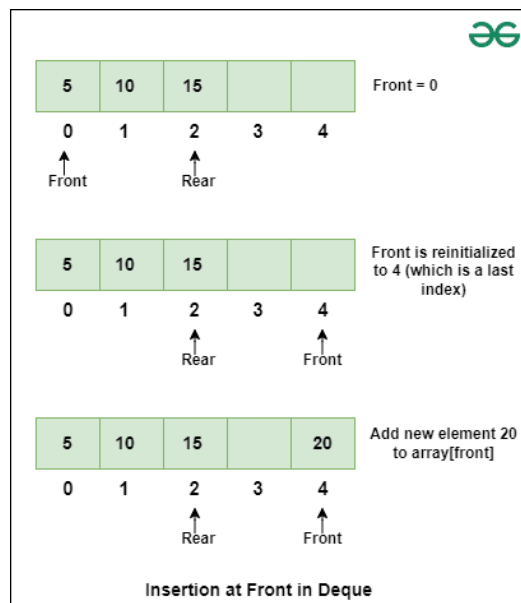
1. Declare an array(to be used as a deque) of size N.
2. Set two pointers at first position i.e. front = -1 and rear = 0



Deque

1. Insertion at Front in Deque in C

To insert an element at the front end of the deque first, check if the deque is full or not if deque is not full then follow the below approach:

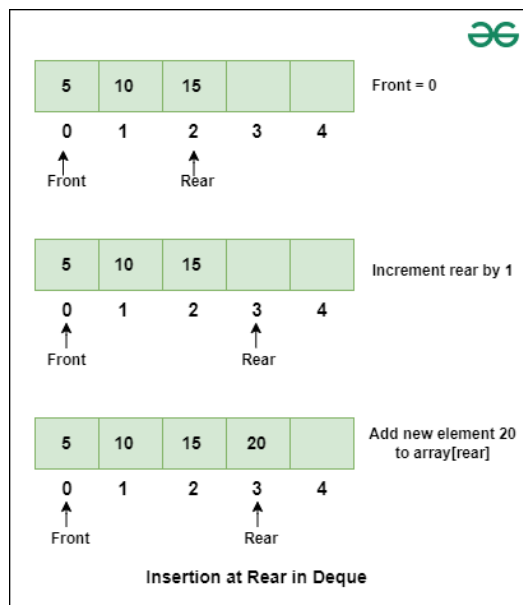


Insert at front in deque

Approach:

- First, check the position of the front in our array.
- If $front < 1$, reinitialize front as the last index of the array i.e. $front = N-1$.
- Then, add new element to $array[front]$.

2. Insertion at Rear in Deque in C



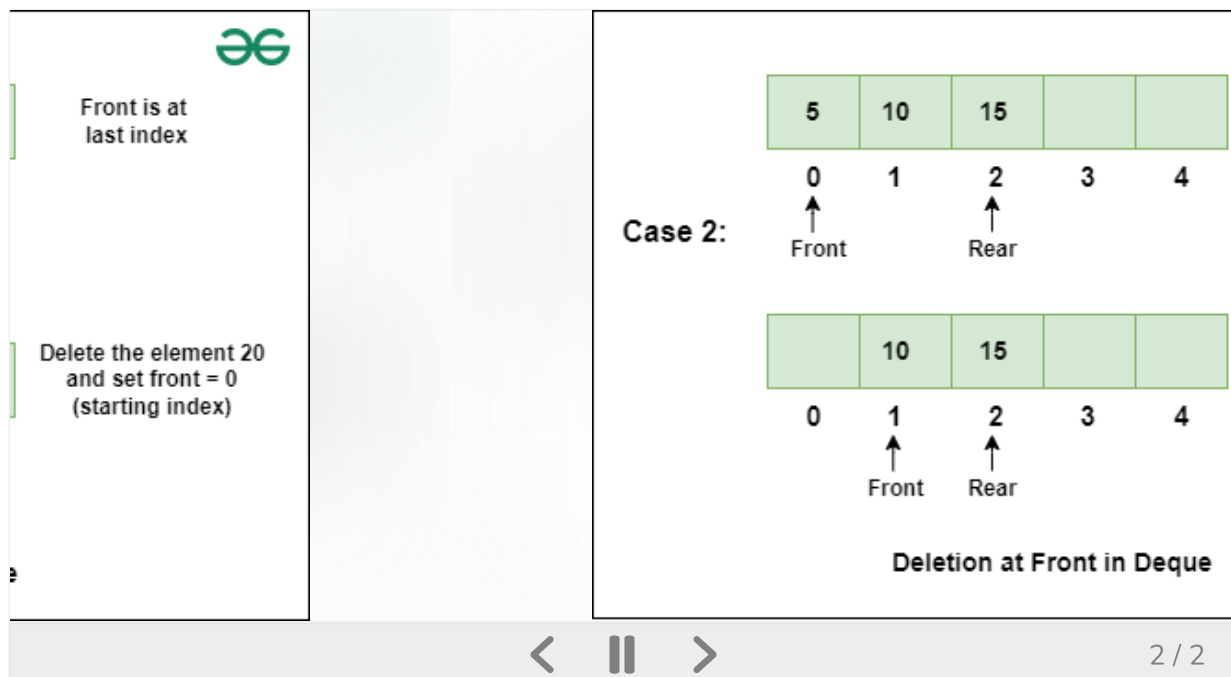
Insert at Rear in Deque

To insert an element at the rear end of the deque, follow the below approach:

Approach:

- First, check if the deque is full or not.
- If the deque is full, reinitialize rear with 0 ($rear = 0$) else increase rear by 1.
- Then, add the element to `array[rear]`.

3. Deletion at Front in Deque in C

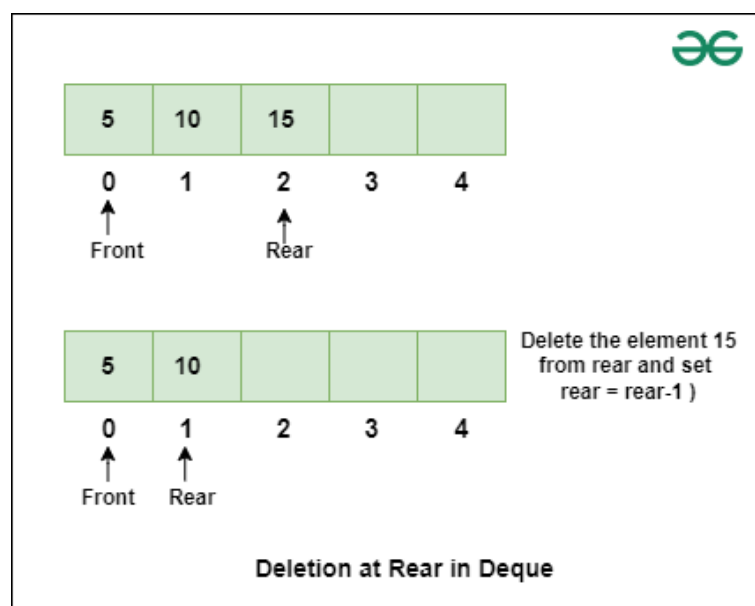


To delete an element at the front end of the deque first, follow the below approach:

Approach:

- First, check if deque is empty or not.
- If the deque is empty ($\text{front} == -1$), then we cannot perform deletion operation. In this condition, we will simply print underflow.
- If deque contains only 1 element ($\text{front} = \text{rear}$), then only one deletion operation can be performed. set $\text{front} = -1$ and $\text{rear} = -1$.
- Else if the front is at the last index ($\text{front} == n-1$), set front at starting index of deque ($\text{front} = 0$).
- If none of the above case exists, just increment front by 1 ($\text{front} = \text{front} + 1$).

4. Deletion at Rear in Deque in C



Delete at Rear in Deque

To delete an element at the rear end of the deque, follow the below approach:

- First, check if the deque is empty or not.
- If the deque is empty ($\text{front} = -1$), then deletion operation cannot be performed and we will print underflow.

- If the deque has only 1 element($front == rear$), we will set $front = -1$ and $rear = -1$.
- If the rear is at the starting index of deque ($rear == 0$) , then set rear to last index ($rear = n-1$).
- If none of the above case exists, just decrement rear by 1 ($rear = rear-1$).

Check if Deque is Empty or Not Empty

To check if the deque is empty simply check the front if $front = -1$, then deque is empty else it is not empty.

Check if Deque is Full or Not Full

To check if the deque is full simply check the below conditions:

- If $front == 0$ and $rear == n-1$, then deque is Full
- If $front == rear + 1$, then also deque is Full.

C Program to Implement Double-Ended Queue

The below program demonstrates all the major operations of a double-ended queue: insertion (at the front and at the rear), deletion (at the front and at the end), check empty and check full.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5 // Define maximum size of the deque

int deque[MAX];
int front = -1;
int rear = -1;

// Function to check if the deque is full
int isFull() {
    return ((front == 0 && rear == MAX - 1) || (front == rear + 1));
}
```

```

// Function to check if the deque is empty
int isEmpty() {
    return (front == -1);
}

// Function to insert an element at the front of the deque
void insertFront(int key) {
    if (isFull()) {
        printf("Overflow: Unable to insert element at the front.
Deque is full.\n");
        return;
    }

    if (front == -1) { // If deque is initially empty
        front = 0;
        rear = 0;
    } else if (front == 0) {
        front = MAX - 1; // wrap around
    } else {
        front = front - 1;
    }

    deque[front] = key;
    printf("Inserted %d at the front.\n", key);
}

// Function to insert an element at the rear of the deque
void insertRear(int key) {
    if (isFull()) {
        printf("Overflow: Unable to insert element at the rear.
Deque is full.\n");
        return;
    }

    if (rear == -1) { // If deque is initially empty
        front = 0;
        rear = 0;
    } else if (rear == MAX - 1) {
        rear = 0; // wrap around
    } else {
        rear = rear + 1;
    }
}

```



```

    deque[rear] = key;
    printf("Inserted %d at the rear.\n", key);
}

// Function to delete an element from the front of the deque
void deleteFront() {
    if (isEmpty()) {
        printf("Underflow: Unable to delete element from the front.
Deque is empty.\n");
        return;
    }

    int removed = deque[front];

    if (front == rear) { // Deque has only one element
        front = -1;
        rear = -1;
    } else if (front == MAX - 1) {
        front = 0; // wrap around
    } else {
        front = front + 1;
    }

    printf("Deleted %d from the front.\n", removed);
}

// Function to delete an element from the rear of the deque
void deleteRear() {
    if (isEmpty()) {
        printf("Underflow: Unable to delete element from the rear.
Deque is empty.\n");
        return;
    }

    int removed = deque[rear];

    if (front == rear) { // Deque has only one element
        front = -1;
        rear = -1;
    } else if (rear == 0) {
        rear = MAX - 1; // wrap around
    }
}

```

```

    } else {
        rear = rear - 1;
    }

    printf("Deleted %d from the rear.\n", removed);
}

// Function to display the deque
void displayDeque() {
    if (isEmpty()) {
        printf("Deque is empty.\n");
        return;
    }

    printf("Deque elements are: ");
    int i = front;
    while (1) {
        printf("%d ", deque[i]);
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

// Main function to test the operations
int main() {
    insertRear(5);
    displayDeque();

    insertFront(15);
    displayDeque();

    insertRear(25);
    displayDeque();

    deleteFront();
    displayDeque();

    deleteRear();
    displayDeque();
}

```

```
    return 0;  
}
```

Output

```
Inserted 5 at the rear.  
Deque elements are: 5  
Inserted 15 at the front.  
Deque elements are: 15 5  
Inserted 25 at the rear.  
Deque elements are: 15 5 25  
Deleted 15 from the front.  
Deque elements are: 5 25  
Deleted 25 from the rear.  
Deque elements are: 5
```

Time Complexity: $O(1)$, as InsertRear(), InsertFront() , DeleteRear(), DeleteFront() have time complexity of $O(1)$ (Constant).

Auxiliary Space: $O(n)$

Applications of Deque

- It can be used in job scheduling algorithm called A-Steal algorithm.
- It is be used for to store a software application's list of undo operations.
- It can be used to implement both stack and queue.
- It can also be used to store the browsing history that includes recently visited URLs.
- It can be used in graph traversal BFS to store nodes and perform operations such as adding or removing nodes from both ends of the deque.

Advantages of Deque

- It allows us to add and remove elements from both ends, hence providing more flexibility than a regular queue or stack.
- It can be used to implement both stacks and queues.
- It can efficiently provide the functionality of both LIFO and FIFO data structures.
- It is efficient as it takes $O(1)$ time complexity to work with both ends.

- Deques are dynamic in size and can grow and reduce in size dynamically.
- It is cache friendly as deque have cache-friendly contiguous subsequence.

Disadvantages of Deque

- Deque is less memory efficient than a normal queue.
- It can cause synchronization issues multi-thread.
- It may not be supported by all platforms in such we need to implement it manually.
- It has limited functionality as compared to other data structures.
- Not suitable for sorting or searching, as these operations require linear time.

[Comment](#)[More info ▼](#)[Advertise with us](#)[Next Article >](#)[C implementation Double-Ended Queue](#)

Similar Reads

Employee Management System using doubly linked list in C

Design and implement a menu-driven program in C for the below operations on DLL of employee data with fields: SSN, name, department, designation, Salary, Phone Number: Create a DLL of N employee's...

🕒 6 min read

C Program to Implement Queue using Array

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle which means the elements added first in a queue will be removed first from the queue. In this article, we will learn how to...

🕒 6 min read

Producer Consumer Problem in C

Concurrency is an important topic in concurrent programming since it allows us to completely understand how the systems work. Among the several challenges faced by practitioners working with these system...

🕒 5 min read

Menu driven program for all operations on doubly linked list in C

A Linked List is a linear data structure that consists of two parts: one is the data part and the other is the address part. A Doubly Linked List in contains three parts: one is the data part and the other two are the...

🕒 5 min read

C Multiple Choice Questions

C is the most popular programming language developed by Dennis Ritchie at the Bell Laboratories in 1972 to develop the UNIX operating systems. It is a general-purpose and procedural programming...

🕒 4 min read

Priority Queue using array in C++

Priority Queue is an extension of the Queue data structure where each element has a particular priority associated with it. It is based on the priority value, the elements from the queue are deleted. Operations...

🕒 4 min read

Commonly Asked C Programming Interview Questions | Set 3

Q.1 Write down the smallest executable code? Ans. main is necessary for executing the code. Code is C
void main() { } Output Q.2 What are entry control and exit control loops? Ans. C support only 2 loops:...

🕒 5 min read

Handling multiple clients on server with multithreading using Socket Programming ...

This tutorial assumes that the reader has a basic knowledge of socket programming, i.e has a familiarity with basic server and client models. In the basic model, the server handles only one client at a time,...

🕒 6 min read

Stack and Queue C/C++ Programs

The stack and queue are popular linear data structures with a wide variety of applications. The stack follows LIFO (Last In First Out) principle where the data is inserted and extracted from the same side. O...

🕒 3 min read

Zig Zag Level order traversal of a tree using single array

Write a function to print spiral order traversal of a tree. For below tree, function should print 1, 2, 3, 4, 5, 6, 7. We have discussed naive approach and two stack based approach in Level Order with recursion an...

🕒 9 min read
